Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

a) Write a C code to create a process using fork() system call.

b) Read the user input as shell command.

c) Use a system call exec() to execute the shell command.

e) Display the PIDs of parent and child processes.

```
root@LAPTOP-4H877K4U:~# nano os.c
root@LAPTOP-4H877K4U:~#
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if(pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n\n", command);

        execlp(command, command, NULL);

        printf("Execution failed!\n");
    }

    else
```

```
    {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

1)Then press ctrl+oenter ctrl+x2)then gcc filename.c -o filename

```
root@LAPTOP-4H877K4U:~# nano os.c
root@LAPTOP-4H877K4U:~# gcc os.c -o os
root@LAPTOP-4H877K4U:~#
```

3)then ./filename

```
root@LAPTOP-4H877K4U:~# nano os.c
root@LAPTOP-4H877K4U:~# gcc os.c -o os
root@LAPTOP-4H877K4U:~# ./os
Enter a Linux command:
```

4)enter the linux command

```
root@LAPTOP-4H877K4U:~# ./os
Enter a Linux command: ls

Parent Process
Parent PID : 564
Child PID  : 568

Child Process
Child PID : 568
Parent PID: 564

Executing command: ls

Directory  OSSP  fil1  file  os  os.c  process  process.c  prog  prog.c  prog1  prog1.c  program.c
```

```
Child process completed.
root@LAPTOP-4H877K4U:~#root@Pranay:~#
```

Child process completed.

Task 2

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.
An operating system is system software that controls all the hardware and software resources of a computer. Users do not communicate directly with the hardware. Instead, the operating system acts as a bridge between the user, applications, and the hardware
Linux Commands Used

1. uname

The uname command is used to display information about the Linux operating system. It provides details such as the operating system name, kernel version, machine architecture, and hostname. This command helps us identify the system on which Linux is running.

```
root@LAPTOP-4H877K4U:~# uname
Linux
root@LAPTOP-4H877K4U:~#
```

2. lscpu

The lscpu command displays detailed information about the processor. It shows the CPU model, number of cores, number of threads, CPU architecture, and processor speed. This information helps us understand the hardware capabilities of the computer.

```
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             39 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      16
  On-line CPU(s) list:       0-15
Vendor ID:                   GenuineIntel
  Model name:                13th Gen Intel(R) Core(TM) i7-13620H
    CPU family:              6
    Model:                   186
```

```
    Thread(s) per core:      2
    Core(s) per socket:      8
    Socket(s):               1
    Stepping:                2  Core(s) per socket:     6
  Socket(s):             1
  Stepping:              4
  BogoMIPS:              4992.00
  Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good
nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx sse3 fma cx16
pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand
hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid
ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb
sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnmi umip waitpkg gfni vaes vpclmulqdq rdpid
movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capabilities
Virtualization features:
  Virtualization:        VT-x
  Hypervisor vendor:     Microsoft
  Virtualization type:   full
Caches (sum of all):
  L1d:                   288 KiB (6 instances)
```

3. ps

The ps command lists the processes that are currently running in the system. It displays the Process ID (PID), Parent Process ID (PPID), user name, and the command associated with each process. This command is useful for monitoring and managing processes.

```
root@LAPTOP-4H877K4U:~# ps
    PID TTY          TIME CMD
    577 pts/0    00:00:00 bash
    604 pts/0    00:00:00 ps
root@LAPTOP-4H877K4U:~#
```

4. top

The top command provides a real-time view of the system's performance. It continuously displays CPU usage, memory usage, running processes, and system load. It is commonly used to identify programs that consume a large amount of system resources.

```
top - 06:54:41 up 11 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  22 total,   1 running,  21 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7733.6 total,   7188.0 free,    534.3 used,    162.2 buff/cache
```

```
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7199.4 avail MemMiB Swap:   2048.0 total,   2048.0
free,      0.0 used,   7282.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   21532  13044   9724 S   0.0   0.2   0:00.53 systemd
      2 root      20   0    3180   2200   2072 S   0.0   0.0   0:00.01 init-systemd(Ub
      6 root      20   0    3196   2128   2008 S   0.0   0.0   0:00.00 init
     52 root      19  -1   50448  16000  14840 S   0.0   0.2   0:00.29 systemd-journal
     97 root      20   0   25380   6788   5168 S   0.0   0.1   0:00.26 systemd-udevd
    165 systemd+  20   0   21344  13232  11108 S   0.0   0.2   0:00.10 systemd-resolve
    166 systemd+  20   0   91036   7988   7020 S   0.0   0.1   0:00.08 systemd-timesyn
    172 root      20   0    4244   2680   2428 S   0.0   0.0   0:00.00 cron
    173 message+  20   0    9528   5308   4612 S   0.0   0.1   0:00.06 dbus-daemon
    180 root      20   0   17980   8856   7812 S   0.0   0.1   0:00.05 systemd-logind
    186 root      20   0 1756104  13112  10888 S   0.0   0.2   0:00.10 wsl-pro-service
    193 syslog    20   0  222516   5988   4624 S   0.0   0.1   0:00.07 rsyslogd
    201 root      20   0    3124   1976   1836 S   0.0   0.0   0:00.00 agetty
    213 root      20   0  107032  23288  13836 S   0.0   0.3   0:00.10 unattended-upgr
    300 root      20   0    6672   4496   3728 S   0.0   0.1   0:00.01 login
    343 root      20   0   20320  11580   9452 S   0.0   0.1   0:00.09 systemd
    344 root      20   0   21168   3516   1804 S   0.0   0.0   0:00.00 (sd-pam)
    368 root      20   0    6080   5276   3652 S   0.0   0.1   0:00.00 bash
    430 root      20   0    3188   1112    980 S   0.0   0.0   0:00.00 SessionLeader
    432 root      20   0    3204   1252   1108 S   0.0   0.0   0:00.07 Relay(435)
    435 root      20   0    6080   5352   3672 S   0.0   0.1   0:00.03 bash
    504 root      20   0    9280   5376   3224 R   0.0   0.1   0:00.00 top
```